String & String Builder in Java

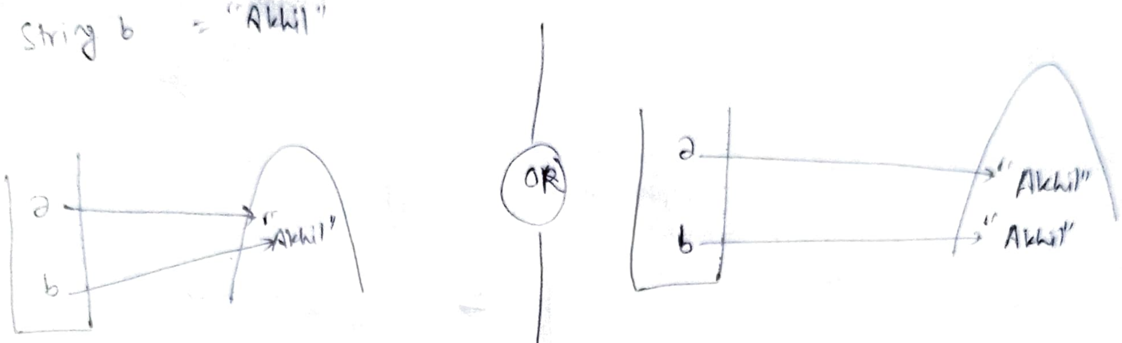
datatype
↓
String name = "Akhilesh Gupta";
ref — name; Object — "Akhilesh Gupta"

System.o.p (name);

| Everything start with capital latter is a class |

String a = "Akhil"

String b = "Akhil"



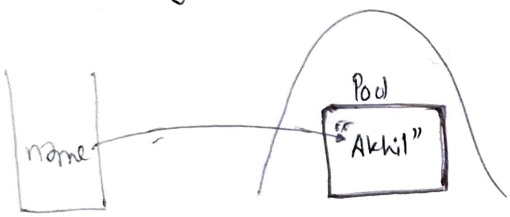
Ⓐ ✓

Ⓑ ✗

| Concept 1 |

| String Pool | ← Separate memory structure inside heap

String name = "Akhil"



Need for Pool ?

No need to create another objects

Changing ~~boy~~ name will it change for b

Not changing → | Immutability |
↓
| For security reason |

| You can't change object | ✓

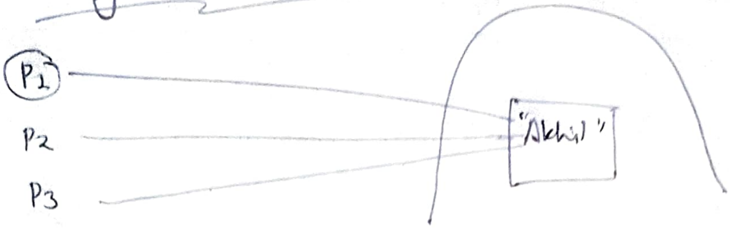
String a = "Akhil"
S.out(a);
a = "Gupta" — Changing a ✗ / Creating new object ✓
Sout (a);

How Akhil changes to Gupta ?

O/P

| Akhil
Gupta |

Why Immutability?



If P1 changes it name

Name for all will change

Comparision of Strings

① == Method

a ⟶ "Akhil"
b ⟶ "Akhil"

false

a, b ⟶ Akhil

true

Check reference variable pointing to same object.

- In Pool: true
- Out Pool: false

② .equals Method

↓

Check for Values Only

- In Pool: true
- Out of Pool: true

String Concatnation

S.O.P('a' + 'b')

"a" + "b" = ab

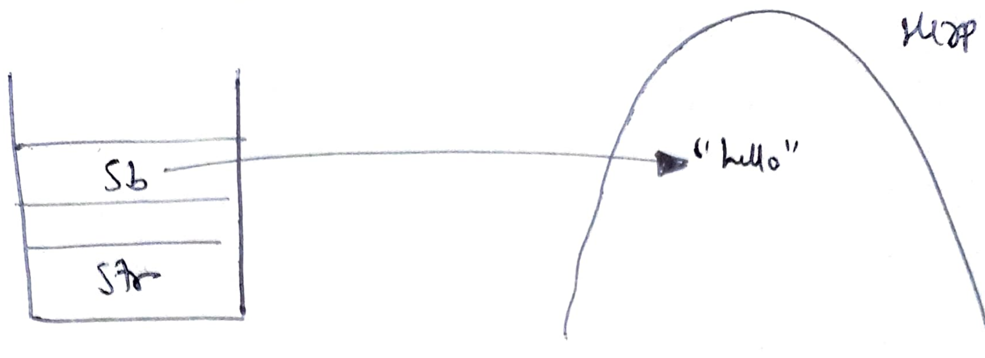
'a' + 3 =

Integer + String

↓

Converted to Integer that will call toString

String Builder



//Declare

```
String Builder  Sb =  new String Builder("Tony");
   S.o.up  (Sb);
```

reversing String using string builder
System.out.println(Sb.reverse());

String Methods

① Substring() Method
Substring(int start, int end); start ↓ Inclusive end ↓ exclusive

② trim() Method
↓
remove blank spaces

toUpperCase & toLowerCase
(Str. ↓) →

③ replace(char oldchar, char newChar)
S.o.up.(Str.replace('x', 'y')) x replace by y

④ indexOf() Method → Str.indexOf("x");

⑤ charAt() Method ⑥ length() Method

⑦ equals() metho ⑧ contains() Method

⑨ StartsWith() Method ⑩ endsWith() Method